FINAL YEAR ENGINEERING PROJECT

Smart Medicine Availability & Intelligent Janaushadhi Recommendation System

INTERVIEW PREPARATION

Project	Smart Medicine Availability & Intelligent Janaushadhi Recommendation System
Document	INTERVIEW PREPARATION
Version	v1.0.0
Date	02 July 2026

React & Frontend Interview Questions with Answers

BEGINNER LEVEL

Q: What is React?

React is a JavaScript library for building user interfaces. It was created by Facebook (Meta). Instead of manually updating the DOM, you describe what the UI should look like for a given state, and React updates the DOM efficiently using its Virtual DOM system.

In this project: The entire frontend — all pages, components, navigation — is built with React.

Q: What is a component?

A component is a JavaScript function that returns JSX (HTML-like syntax). Components are the building blocks of a React application — you compose them together to build pages.

In this project: `Button`, `Badge`, `SearchResultCard`, `Sidebar`, `LoginPage` are all components.

Q: What is JSX?

JSX is a syntax extension for JavaScript that looks like HTML. It allows you to write markup inside JavaScript files. Babel/Vite transpiles JSX to `React.createElement()` calls.

```
// JSX
const element = <h1 className="title">Hello</h1>

// What it compiles to
const element = React.createElement('h1', { className: 'title' }, 'Hello')
```

Q: What are props?

Props (properties) are inputs passed from a parent component to a child component. They are read-only — a child cannot modify its props. Think of them as function parameters.

```
<Badge variant="success" size="sm">In Stock</Badge>
// variant="success" and size="sm" are props
```

Q: What is state?

State is data that lives inside a component. When state changes, React re-renders the component. `useState` is the hook for managing state.

```
const [isSaved, setIsSaved] = useState(false)
// isSaved is state. setIsSaved changes it.
```

Q: What is the Virtual DOM?

The Virtual DOM is a lightweight JavaScript representation of the real DOM. When state changes, React creates a new Virtual DOM, diffs it with the previous one, and only updates the changed parts in the real DOM. This is more efficient than re-rendering the entire page.

Q: What is the difference between `class` and `className` in JSX?

In HTML you write `class="btn"`. In JSX you write `className="btn"` because `class` is a reserved keyword in JavaScript.

Q: What is `useState`?

`useState` is a React hook that adds state to a functional component. Returns an array of `[currentValue, setter]`.

```
const [count, setCount] = useState(0)
setCount(count + 1)   // update state
```

Q: What is `useEffect`?

`useEffect` runs code after a component renders. Used for side effects: API calls, event listeners, timers, DOM manipulation.

```
useEffect(() => {
  // Runs when `id` changes
  fetchMedicine(id)
}, [id])
```

Q: What is a key prop in lists?

When rendering lists with `.map()`, React needs a unique `key` prop to track which items changed.

```
{medicines.map((m) => (
  <MedicineCard key={m.id} medicine={m} />
))}
```

Without keys: React may re-render the entire list. With keys: React only updates changed items.

INTERMEDIATE LEVEL

Q: What is Context API and why was it used in this project?

Context API allows sharing state across many components without prop drilling. In this project:

- `AuthContext` — provides `currentUser`, `login`, `logout` to any component
- `ThemeContext` — provides `theme`, `toggleTheme`

Without Context, you'd pass `currentUser` as a prop through: App → AppRouter → UserLayout → TopBar → Avatar — 4 levels of unnecessary prop passing.

Q: What is the difference between controlled and uncontrolled components?

- **Controlled:** React state is the source of truth. Every keystroke updates state. `useState` driven.
- **Uncontrolled:** The DOM is the source of truth. React reads values with a ref when needed.

This project uses **React Hook Form** which uses uncontrolled inputs internally for performance, but provides a controlled-like API.

Q: What are custom hooks?

Custom hooks are functions that start with `use` and call other React hooks. They extract reusable stateful logic from components.

```
// useDebounce is a custom hook
const debouncedQuery = useDebounce(searchInput, 400)
```

In this project: `useDebounce`, `useLocalStorage`, `useOnlineStatus`, `useSessionGuard`.

Q: What is lazy loading and why did you use it?

Lazy loading delays loading a module until it is needed. In this project, all pages are lazy-loaded:

```
const UserDashboard = lazy(() => import('../pages/dashboard/UserDashboard'))
```

Why: Without lazy loading, the main bundle was ~963KB. With lazy loading, it is 464KB — 50% smaller. Users see the home page much faster because they don't download the admin portal code until they navigate there.

Q: What is the difference between `navigate()` and `<Link>`?

- `<Link>` — declarative navigation in JSX (like `<a>` but SPA-friendly)
- `navigate()` — programmatic navigation from JavaScript code (e.g., after form submission)

```
// After login success
const navigate = useNavigate()
navigate('/dashboard')

// In JSX
<Link to="/search">Search Medicines</Link>
```

Q: What is React Router and what is `<Outlet />`?

React Router is a library for client-side routing in React SPAs. `<Outlet />` is a placeholder in layout components where the matched child route renders.

When the user visits `/dashboard`:

1. `UserLayout` renders (Sidebar + TopBar)
2. Inside `<Outlet />`, `UserDashboard` renders

Q: What is React Hook Form and why is it better than controlled inputs?

React Hook Form manages form state using uncontrolled inputs and refs. It avoids re-rendering the component on every keystroke — only re-renders on form submit or when validation errors change. This is significantly more performant for large forms like the registration form.

Q: What is Zod and why was it used?

Zod is a TypeScript-first schema validation library. In this project, `authSchemas.js` contains Zod schemas for all auth forms. `zodResolver(schema)` connects Zod to React Hook Form.

Benefits:

- Schema defined once, reusable
- Type inference (TypeScript-ready)
- Composable (email field reused in login + register)

Q: What is TanStack React Query?

React Query is a server-state management library. It handles caching, loading states, background refetching, and query invalidation. In this project it is configured with `staleTime: 1 minute`, `gcTime: 5 minutes`, `retry: 1`.

When backend is integrated, it replaces manual `useState + useEffect + fetch` patterns.

Q: What is Axios and why was it used instead of `fetch`?

Axios is an HTTP client library. Used over native `fetch` because:

- Automatic JSON parsing (no `.json()` call needed)
- Request/response interceptors (JWT token, 401 handling)
- Timeout support
- Better error objects

Q: How does JWT authentication work in this frontend?

1. Login → store `accessToken` + `refreshToken` in localStorage
2. Every request → `axiosClient` attaches `Authorization: Bearer {token}`
3. Token expires → backend returns 401
4. `axiosClient` interceptor catches 401 → clears session → redirects to `/session-expired`

ADVANCED LEVEL

Q: What is code splitting and how was it implemented?

Code splitting breaks the JavaScript bundle into smaller chunks that load on demand. Implemented via `React.lazy()` + dynamic `import()`. Vite handles the chunk splitting automatically.

Result: 43 chunks instead of 1 large bundle. Initial load is fast; other pages load on first visit.

Q: What is `useCallback` and when is it needed?

`useCallback` memoises a function reference between renders. Used when:

1. The function is a dependency in `useEffect`
2. The function is passed as a prop to a memoised child component

In `AuthContext`, all action functions (`login`, `logout`, `register`) use `useCallback` to prevent them from being recreated on every render, which would trigger useEffect re-runs in consuming components.

Q: What is an Error Boundary?

An Error Boundary is a React class component that catches JavaScript errors in the component tree below it and renders a fallback UI instead of crashing.

`AppErrorBoundary` in this project wraps the entire application. If any component throws, `ServerErrorPage` renders instead of a blank white screen.

Error Boundaries must be class components because they use `componentDidCatch` lifecycle method (no hooks equivalent exists).

Q: What is the difference between `localStorage` and React state?

	React State	localStorage
Persistence	Cleared on page refresh	Survives page refresh
Access	Only in React components	From anywhere
Reactivity	Triggers re-render	Does not trigger re-render

In this project: JWT tokens and user object are in localStorage (persists sessions). UI state (isLoading, authError) is in React state.

Q: Why is the navigation config in a separate file (`navConfig.js`)?

Separation of concerns. If nav items were hardcoded in `Sidebar.jsx`:

- Adding a nav item requires editing the layout file
- Risk of breaking the layout

With `navConfig.js`:

- Add a nav item in the constants file
- Sidebar renders whatever is in the config
- Zero risk of breaking the layout component

Q: What is the `state` pattern in `ProtectedRoute`?

```
<Navigate to="/login" state={{ from: location }} replace />
```

When an unauthenticated user tries to visit `/dashboard`, they are redirected to `/login` but the original destination is saved in `state`. After login, the app can redirect them back to `/dashboard` instead of always going to the default route. Better UX.

Q: What is a barrel export (`index.js`)?

A file that re-exports everything from a folder:

```
// components/ui/index.js
export { default as Button } from './Button'
export { default as Badge } from './Badge'
```

Consumers import cleanly:

```
import { Button, Badge } from '../components/ui'
```

```
// Instead of:
import Button from '../components/ui/Button'
import Badge from '../components/ui/Badge'
```

Q: How does the online/offline detection work?

`useOnlineStatus` hook listens to browser `online` and `offline` events:

```
window.addEventListener('online', () => setIsOnline(true))
window.addEventListener('offline', () => setIsOnline(false))
```

`App.jsx` consumes this hook and renders `<OfflinePage>` when offline. No backend involved — pure browser API.

Q: What is the Single Responsibility Principle as applied in this project?

Each file does one job:

- `axiosClient.js` — only HTTP configuration
- `authService.js` — only auth API calls
- `AuthContext.jsx` — only auth state management
- `LoginPage.jsx` — only the login UI
- `validators.js` — only validation functions

No file does everything. This makes each file testable and changeable in isolation.